

1. Introduction to Loops

In programming, sometimes we need to repeat the same task multiple times.

For example:

- Printing numbers from 1 to 100
- Displaying student names
- Repeating calculations
- Generating patterns

Writing the same code repeatedly is inefficient.

To solve this problem, programming languages provide **loops**.

A **loop** is used to execute a block of code repeatedly until a condition is satisfied.

2. Why Loops are Important

Loops help programmers:

- Reduce repetitive code
 - Save time
 - Improve efficiency
 - Make programs shorter and cleaner
 - Automate repetitive tasks
-

Example Without Loop

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

This method becomes difficult for large repetitions.

Example With Loop

```
for i in range(1, 6):
    print(i)
```

This single loop prints numbers from 1 to 5.

3. What is a for Loop?

A **for loop** is used to repeat a block of code for a fixed number of times.

It is commonly used when we know:

- How many times the loop should run
 - The sequence of values to iterate through
-

Syntax of for Loop

```
for variable in sequence:  
    statements
```

Example

```
for i in range(5):  
    print(i)
```

Output

```
0  
1  
2  
3  
4
```

Explanation

- i is the loop variable
 - range(5) generates numbers from 0 to 4
 - The loop executes 5 times
-

4. Working of for Loop

The flow of execution:

1. Loop starts
2. First value is assigned to variable
3. Statements execute
4. Next value is assigned
5. Process repeats
6. Loop stops when sequence ends

5. The range() Function

The range() function generates a sequence of numbers.

It is mostly used with for loops.

Syntax

```
range(start, stop, step)
```

Parameters

Parameter Meaning

start	Starting value
stop	Ending value (excluded)
step	Increment or decrement

6. Types of range() Usage

6.1 range(stop)

```
for i in range(5):  
    print(i)
```

Output:

0 1 2 3 4

6.2 range(start, stop)

```
for i in range(1, 6):  
    print(i)
```

Output:

1 2 3 4 5

6.3 range(start, stop, step)

```
for i in range(1, 10, 2):  
    print(i)
```

Output:

1 3 5 7 9

7. Flow of Execution in for Loop

Example:

```
for i in range(3):  
    print("Hello")
```

Execution:

- `i = 0` → prints Hello
 - `i = 1` → prints Hello
 - `i = 2` → prints Hello
 - Loop ends
-

8. Real-Life Examples of Loops

Loops are used in:

- ATM transaction systems
 - Sending notifications
 - Printing reports
 - Game development
 - Data processing
 - Websites and mobile apps
 - Artificial Intelligence systems
-

9. Nested Loops

A loop inside another loop is called a **nested loop**.

Example

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Output

```
0 0
0 1
1 0
1 1
2 0
2 1
```

Explanation

- Outer loop runs 3 times
 - Inner loop runs 2 times for each outer loop iteration
-

10. Introduction to Number Patterns

Pattern programs are one of the best ways to learn loops.

Patterns help students:

- Improve logic building
 - Understand nested loops
 - Learn row-column structures
-

Basic Pattern Structure

Most pattern programs use:

- Outer loop → controls rows
 - Inner loop → controls columns
-

11. Types of Number Patterns

Common pattern types:

- Number patterns
 - Star patterns
 - Pyramid patterns
 - Reverse patterns
 - Triangle patterns
-

12. Simple Number Pattern Programs

Pattern 1: Print Numbers in Single Line

```
for i in range(1, 6):  
    print(i)
```

Output

```
1  
2  
3  
4  
5
```

Pattern 2: Horizontal Number Pattern

```
for i in range(1, 6):  
    print(i, end=" ")
```

Output

```
1 2 3 4 5
```

Pattern 3: Square Number Pattern

```
for i in range(5):  
    for j in range(5):  
        print(j + 1, end=" ")  
    print()
```

Output

```
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5
```

Pattern 4: Increasing Triangle Pattern

```
for i in range(1, 6):  
    for j in range(1, i + 1):  
        print(j, end=" ")  
    print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Pattern 5: Reverse Triangle Pattern

```
for i in range(5, 0, -1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

13. Star Patterns

Star Square Pattern

```
for i in range(5):
    for j in range(5):
        print("*", end=" ")
    print()
```

Output

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Star Triangle Pattern

```
for i in range(1, 6):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

14. Pyramid Patterns

Simple Pyramid Pattern

rows = 5

```
for i in range(rows):
    for j in range(rows - i - 1):
        print(" ", end="")

    for k in range(2 * i + 1):
        print("*", end="")

    print()
```

Output

```
  *
 ***
*****
*****
*****
```

15. Common Errors in Loops

1. Incorrect Indentation

Wrong:

```
for i in range(5):
print(i)
```

Correct:

```
for i in range(5):
    print(i)
```

2. Wrong Range Values


```
range(1, 5)
```

Prints:

```
1 2 3 4
```

The stop value is excluded.

3. Infinite Loops

Incorrect loop conditions may cause loops to run forever.

16. Advantages of Loops

- Reduces code repetition
 - Makes code cleaner
 - Saves development time
 - Improves efficiency
 - Useful for automation
 - Handles large datasets easily
-

17. Real-Life Applications

Loops are widely used in:

- Banking software
- AI systems
- Data analysis
- Machine learning
- Websites
- Games
- Mobile apps
- Attendance systems
- Report generation